provadis Hochschule

WAB

Provadis School of International Management and Technology

Analyzing the Correlation Between Cyclomatic Complexity and Code Quality in Python Projects

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Agile Softwareentwicklung und Softwaretechnik
Reviewer: Philip Ceh

04.05.2026

Abstract

Abstrakt

Contents

List of Figures

List of Tables

Glossary

Abbreviations

1. Introduction

1.1. Background

2. Methods

2.1. Design

3. Results

4. Discussion

4.1. Limitations

5. Conclusion

5.1. Future Work

References

AI Declaration

The usage of AI tools within this project is documented here. I declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	I want you to rewrite my template from LaTeX to Typst, and make it look like it did before.	Recreated the template in Typst, maintaining the original appearance.
GitHub Copilot 2	In Code/, write a Python script that finds the top most depended-upon Python packages and analyzes the correlation between function cyclomatic complexity and bug-fix frequency over an adjustable time window, with plots and tables.	Created the analysis pipeline.
GitHub Copilot 3	Use pip to install any required packages, not conda.	Added pip dependency setup.
GitHub Copilot 4	Correct the package discovery so the ranking reflects dependency counts and the analysis uses real source repositories rather than release repositories.	Fixed discovery and repo resolution.
GitHub Copilot 5	Make the analysis practical to run by adding adjustable bounds for the mined recent commit history.	Added bounded commit mining.
GitHub Copilot 6	Use ProcessPoolExecutor to accelerate the script.	Parallelized package mining.
GitHub Copilot 7	Add compact logging and terminal progress bars, show phases such as cloning, mining, SZZ, and plotting, and remove Typst table generation.	Added logs and progress bars; removed Typst exports.
GitHub Copilot 8	Fix the parser warning noise produced while mining repositories.	Suppressed parser warnings.

System	Prompt	Usage
GitHub Copilot 9	Fix the Windows UnicodeDecodeError that occurs while reading subprocess output during mining.	Hardened subprocess decoding.
GitHub Copilot 10	Implement flexible caching so repeated runs can reuse already mined information while still respecting parameter changes.	Added parameter-aware mining cache.
GitHub Copilot 11	Add analysis support for native code too and update the progress bars more often.	Added mixed-language analysis; increased progress updates.
GitHub Copilot 12	Use all languages recognized by lizard instead of restricting the non-Python analysis to C-family files.	Expanded lizard language coverage.
GitHub Copilot 13	Add a CLI option to restrict the analysis to Python files only while keeping mixed-language analysis as the default.	Added Python-only analysis mode.
GitHub Copilot 14	Refactor the analysis script into helper modules and keep the main script as the orchestration entrypoint.	Reorganized the analysis code into helper modules.
GitHub Copilot 15	Replace the PyDriller repository mining implementation with direct Git CLI queries and keep the SZZ analysis working on top of that path.	Replaced PyDriller-based mining with Git CLI mining and SZZ integration.
GitHub Copilot 16	Improve the SZZ output by separating unresolved cases and adding rename-aware matching for moved files and renamed functions.	Added SZZ attribution categories and rename-aware matching.
GitHub Copilot 17	Rename the stale non-Python parser helper so it reflects the current lizard-based multi-language behavior.	Renamed the non-Python parser helper for clarity.
GitHub Copilot 18	Add NumPy-style docstrings to the important analysis, mining, and reporting functions.	Added NumPy-style docstrings to key functions.
GitHub Copilot 19	Rewrite the repository README so it reads like a cohesive project README rather than patchwork.	Reworked the README structure and usage documentation.

System	Prompt	Usage
GitHub Copilot 20	Make plotting reusable from precomputed results, add richer reporting outputs, and export grouped raw JSON.	Added plot-only rendering, extra summaries and plots, and raw JSON output.
GitHub Copilot 21	Persist commit dates for bug-fixing and bug-introducing commits and add a time-series plot for bug-fix commits.	Added commit-date export and bug-fix timeline reporting.
GitHub Copilot 22	Replace the many CLI flags with a TOML config file, keep only –cfg in the CLI, and make rename-matching thresholds configurable.	Introduced TOML-based configuration and configurable rename matching.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, 04.05.2026

Rubin Chempananickal James